```
end

def unavailable?
  held? || purchased?
end
```

Now we need to make the context available to the rest of the app.

Using the Context

In React, you use a context by making a JSX element called a *provider*. You can see our provider in the returned JSX at the end of the file. We create a JSX element called `VenueContext.Provider` and place our existing `Venue` component inside it. We pass one prop to the provider, called `value`, which is an object with `{state, dispatch}`. That object syntax defines an object where the key and value have the same name, in other words, the object being passed to the prop is `{state: state, dispatch: dispatch}`.

As keys, `state` and `dispatch` are defined by the `IsVenueContext` interface. The `state` and `dispatch` values, however, came from the previous line's call to the `useReducer` hook, `const [state, dispatch] = React.useReducer(venueReducer, fetchState(props))`.

Let's walk through our app to see how the components change now that the data is managed by the context.

Venue

The `Venue` component no longer needs to take properties in just to pass them along. Without having to pass properties or handle data, the `Venue` component gets a lot simpler:

chapter_11/01/app/javascript/components/venue.tsx

```
import * as React from "react"
import Subtotal from "./subtotal"
import VenueBody from "./venue_body"
import VenueHeader from "./venue_header"

export const Venue = (): React.ReactElement => {
  return (
```